



TUBAF

Die Ressourcenuniversität.
Seit 1765.

PERSONAL PROGRAMMING PROJECT

SS2026

Python/NGSolve Implementation of a Thermodynamically Consistent
Neural Network as a Hyperelastic Material Surrogate
(Multiscale Homogenization of PDMS-Glass Bead Composites)

Supervisor: Dr.-Ing. habil. Martin Abendroth

Vignesh Ravi
Matriculation No: 71807

Motivation & The Multiscale Bottleneck

The Material Challenge:

- Soft matrix (PDMS) $\mu_{PDMS} \approx 2 \text{ MPa}$, near-incompressible.
- Rigid inclusions (Glass): $E_{Glass} \approx 70,000 \text{ MPa}$
- Result: Extreme microstructural stiffness contrast leading to highly non-linear macroscopic hyperelasticity.

The Multiscale Bottleneck (Concurrent vs. Offline Surrogate):

- Scale separation: $l_{micro} \ll L_{macro}$
- **Concurrent Multiscale Simulation:** Requires dynamically solving a microscopic Boundary Value Problem (BVP) at every macroscopic integration point during simulation. Time complexity of $\mathcal{O}(N_{macro} \times N_{micro})$ makes this computationally prohibitive.
- **The Goal (Offline Homogenization):** Decouple the scales. Formulate a parameterized Input-Convex Neural Network (pICNN) to map $\bar{\mathbf{F}} \rightarrow (\bar{\mathbf{P}}, \bar{\mathbf{A}})$ offline, and implement it as a highly efficient custom Material Routine (UMAT) inside an NGSolve Newton-Raphson solver.
- **Two-Phase Strategy:** Phase 1 (Proof of Concept): Establish the PyTorch-NGSolve UMAT bridge using a baseline Neo-Hookean model on a single element.
- Phase 2 (Multiscale): Once verified, generate complex RVE data and train the advanced parameterized Input-Convex Neural Network (pICNN).

Legend:

μ, E : Shear modulus (PDMS) and Young's modulus (Glass).

l_{micro}, L_{macro} : Characteristic length scales of the RVE and the macroscopic component.

N_{micro}, N_{macro} : Number of degrees of freedom / integration points at each respective scale.

$\bar{\mathbf{F}}, \bar{\mathbf{P}}, \bar{\mathbf{A}}$: Macroscopic Deformation Gradient, 1st Piola-Kirchhoff Stress, and 4th-Order Tangent Stiffness.



Microscopic Homogenization & Data Generation

Constitutive Models for RVE Generation:

- PDMS Matrix (Neo-Hookean): $\Psi_{PDMS} = \frac{\mu}{2} (I_1 - 3) - \mu \ln J + \frac{\Lambda}{2} (\ln J)^2$
- Glass Beads (Rigid Penalty): $E \rightarrow \infty$ approximation inside Netgen.

Kinematics & Periodic Boundary Conditions (PBCs):

- Fluctuation field $\tilde{\mathbf{w}}$ must be periodic to prevent boundary stiffening:
- $\mathbf{u}^+ - \mathbf{u}^- = (\bar{\mathbf{F}} - \mathbf{I})(\mathbf{x}^+ - \mathbf{x}^-)$

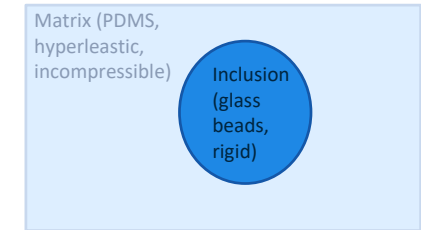
Hill-Mandel Volume Averaging:

- Deformation: $\bar{\mathbf{F}} = \frac{1}{V_{RVE}} \int_{\partial V} \mathbf{x} \otimes \mathbf{n} dA$
- Stress: $\bar{\mathbf{P}} = \frac{1}{V_{RVE}} \int_V \mathbf{P}(\mathbf{F}) dV$
- Tangent: $\bar{\mathbf{A}} = \frac{\partial \bar{\mathbf{P}}}{\partial \bar{\mathbf{F}}}$

Handling Rigid Inclusions: Modeled via extreme stiffness penalty ($E_{glass} \gg E_{matrix}$) to enforce near-rigid kinematics within the standard BVP.

Periodic Boundary Conditions: Enforced strictly by utilizing NGSolve's native Periodic FESpaces mapped onto geometrically periodic Netgen meshes.

Macroscopic Tangent ($\bar{\mathbf{A}}$): Extracted via numerical perturbation of the RVE macroscopic deformation gradient (forward finite difference).



Legend:

Ψ_{PDMS} : Microscopic strain energy density of the host matrix.

λ : First Lamé parameter (controlling compressibility).

I_1, J : First kinematic invariant $\text{Tr}(\mathbf{F}^T \mathbf{F})$ and Volume ratio $\det(\mathbf{F})$

$\mathbf{u}^+, \mathbf{u}^-$: Displacement vectors at opposing periodic boundaries.

$\mathbf{x}^+, \mathbf{x}^-$: Reference coordinate vectors at opposing periodic boundaries.

$\mathbf{I}$: Second-order identity tensor.

V_{RVE} : Total volume of the Representative Volume Element.

[1] Holzapfel, G.A. (2000). Nonlinear Solid Mechanics: Neo-Hookean Energy Functional & Finite Deformations

[2] Hill, R. (1963). Elastic properties of reinforced solids: The Hill-Mandel Macrohomogeneity Condition (Volume Averaging). Journal of the Mechanics and Physics of Solids, 11(5), 357-372.

Formulating the pICNN Surrogate

The pICNN Surrogate Model ($\hat{\Psi}$):

- Objective: AI directly approximates the macroscopic strain energy density.

Polyconvexity & Material Stability:

- Function must be Polyconvex (Ball, 1977) to prevent solver divergence.
- Kinematic invariants act as isolated inputs: $\hat{\Psi} = \mathcal{N}(I_1, I_2, v_f)$
- Weight matrices $\mathbf{W}^{(l)}$ are constrained to be non-negative:
- $\mathbf{W}_{\text{constrained}}^{(l)} = \text{softplus}(\mathbf{W}^{(l)}) = \ln(1 + \exp(\mathbf{W}^{(l)}))$
- Result: Guarantees a strictly positive-definite tangent stiffness matrix $\hat{\mathbf{A}}$.*

Thermodynamic Normalization:

- Enforcing a stress-free reference state at $\mathbf{F} = \mathbf{I}$:
- $\widehat{\Psi}_{\text{stable}}(\mathbf{F}) = \widehat{\Psi}_{\text{raw}}(\mathbf{F}) - \widehat{\Psi}_{\text{raw}}(\mathbf{I})$

Sobolev Training & Exact Tensor Calculus

Constitutive Mapping via PyTorch Autograd:

- Bypassing numerical finite difference to avoid truncation error.
- 1st Piola-Kirchhoff Stress: $\hat{\mathbf{P}} = \frac{\partial \hat{\Psi}}{\partial \mathbf{F}}$
- 4th-Order Tangent Stiffness: $\hat{\mathbf{A}} = \frac{\partial^2 \hat{\Psi}}{\partial \mathbf{F}^2}$

Sobolev Loss Formulation:

- Minimizing errors on the energy, gradient, and curvature simultaneously:
- $\mathcal{L}_{\text{total}} = \lambda_{\Psi} \frac{1}{N} \sum \|\Psi - \hat{\Psi}\|_2^2 + \lambda_P \frac{1}{N} \sum \|\mathbf{P} - \hat{\mathbf{P}}\|_F^2 + \lambda_A \frac{1}{N} \sum \sum \|\mathbf{A} - \hat{\mathbf{A}}\|_F^2$

Hyperparameter Weighting:

- $\lambda_A \gg \lambda_{\Psi}$ (Accurate tangent is prioritized to drive optimal NR convergence).

Legend:

$\hat{\Psi}$: AI-predicted macroscopic strain energy density.

$\mathcal{N}$: Neural network functional approximation.

I_2 : Second kinematic invariant.

v_f : Glass bead volume fraction (parameterization variable).

$\mathbf{W}^{(l)}$: Internal weight matrix at hidden layer l .

- 4
- [1] Ball, J. M. (1977). Convexity conditions and existence theorems in nonlinear elasticity. Archive for Rational Mechanics and Analysis, 63(4), 337-403.
 - [2] Amos, B., Xu, L., & Kolter, J. Z. (2017). Input convex neural networks. In International Conference on Machine Learning (ICML) (pp. 146-155). PMLR.
 - [3] Kalina, K. A., Linden, L., Brummund, J., & Kästner, M. (2022). Automated constitutive modeling of isotropic hyperelasticity based on artificial neural networks. Computational Mechanics, 69(1), 213-232.
 - [4] Czarnecki, W. M., et al. (2017). Sobolev training for neural networks. Advances in Neural Information Processing Systems (NIPS), 30.



Monolithic UMAT Implementation in NGSolve

Macroscopic Weak Form (Non-Linear BVP):

- $\delta \mathcal{W} = \int_{\Omega} \hat{\mathbf{P}}(\mathbf{F}(\mathbf{u})) : \delta \mathbf{F} d\Omega - \int_{\partial\Omega} \mathbf{t} \cdot \delta \mathbf{u} dA = 0$

Linearization & The AI-Driven Newton-Raphson Update:

- Global Residual: $\mathbf{R}(\mathbf{u}_k) = \mathbf{f}_{\text{int}}(\mathbf{u}_k) - \mathbf{f}_{\text{ext}}$
- Exact Jacobian / Stiffness: $\mathbf{K}_k = \int_{\Omega} \nabla \delta \mathbf{u} : \hat{\mathbf{A}} : \nabla \Delta \mathbf{u} d\Omega$
- Update Step: $\mathbf{u}_{k+1} = \mathbf{u}_k - \mathbf{K}_k^{-1} \mathbf{R}_k$

Algorithmic Bridge (Python/PyTorch):

- Extract $\mathbf{F}_{\text{local}}$ at all macroscopic integration points.
- Query plCNN(F, v_f) to retrieve $\hat{\mathbf{P}}$ and $\hat{\mathbf{A}}$.
- Voigt Mapping: Flatten 4th-order tensor ($3 \times 3 \times 3 \times 3 \rightarrow 6 \times 6$) for matrix assembly.

Legend:

$\mathcal{L}_{\text{total}}$: Total combined Sobolev training loss.

$\lambda_{\Psi}, \lambda_P, \lambda_A$: Hyperparameter weightings for energy, stress, and tangent optimization (λ_A prioritized).

N: Number of samples in the training batch.

[1] Holzapfel, G.A. (2000). Nonlinear Solid Mechanics: Hyperelastic Weak Form Formulation

[2] Belytschko, T., Liu, W. K., Moran, B., & Elkhodary, K. (2014). Nonlinear Finite Elements for Continua and Structures. John Wiley & Sons: Voigt Matrix Assembly in Finite Elements

[3] Bathe, K.-J. (2019). Finite Element Procedures. Prentice-Hall, Inc., Watertown, MA, 2nd edition: Non-Linear Weak Form & Newton-Raphson Update

SUB-TASK AND WORKFLOW

- Geometry and periodic RVE mesh generation.
- Offline volume averaging and data extraction.
- Sobolev training of the pICNN surrogate.
- Extraction of exact analytical tensors via Autograd.
- Define macroscopic coupled weak forms.
- Monolithic custom Newton-Raphson solution.
- Verification using rigorous mathematical benchmarks.
- Final output and documentation.

TECH STACK & IMPLEMENTATION

This project focuses on formulating a decoupled multiscale pipeline where PyTorch AI directly drives a macroscopic NGSolve finite element simulation.

- Language: Python 3.10
- Netgen / NGSolve: C++ based Python interface for automated RVE generation, FE formulation, and macroscopic global matrix assembly.
- PyTorch: Autograd-enabled tensor library for pICNN architecture, exact tangent derivation, and Sobolev loss optimization:

$$\mathcal{L}_{total} = \lambda_{\Psi} \frac{1}{N} \sum \left\| \Psi - \hat{\Psi} \right\|_2^2 + \lambda_P \frac{1}{N} \sum \left\| \mathbf{P} - \hat{\mathbf{P}} \right\|_F^2 + \lambda_A \frac{1}{N} \sum \sum \left\| \mathbf{A} - \hat{\mathbf{A}} \right\|_F^2$$

- NumPy, SciPy: Voigt mapping operations ($3 \times 3 \times 3 \times 3 \rightarrow 6 \times 6$), numerical tensor flattening.
- Matplotlib: Loss landscape visualization, verification plots, quadratic convergence tracking.

IMPLEMENTATION SCOPE & CODE PROVENANCE

Utilized Frameworks

- Netgen / NGSolve: Native C++ classes utilized for geometry generation and standard finite element space initialization.
- PyTorch Framework: Base nn.Module classes and Adam optimizer loops utilized for standard network training.
- AI Assistants (ChatGPT): Utilized strictly as a coding assistant for syntax debugging, generating standard matplotlib plotting templates, and optimizing numpy array manipulations.

Individual Contributions

- **Microstructural PBC Enforcer:** Custom Python algorithms written to strictly map and enforce kinematic periodicity across the complex Netgen matrix-inclusion interface.
- **Polyconvex Surrogate Architecture:** Custom PyTorch forward passes engineered to isolate invariants, enforce softplus weight constraints, and analytically normalize the thermodynamics.
- **The Macroscopic UMAT Bridge:** The core algorithmic contribution. A custom-written Python Newton-Raphson loop that dynamically extracts integration points, queries the AI surrogate, maps the 4th-order Autograd tangent, and manually assembles the global stiffness matrix.
- **The Diagnostic V-Suite:** Custom mathematical scripts written from scratch to execute the Finite Difference calculus checks and verify asymptotic quadratic convergence.

<https://docu.ngsolve.org/latest/i-tutorials/index.html> – PDMS box and glass sphere

<https://pytorch.org/docs/stable/notes/extending.html> - custom forward passes and use of autograd function

<https://pytorch.org/tutorials/> - Use Optimisation and Building models

Linden et al. (2023) JMPS 179:105363 — ICNN with physics constraints $\Psi = \mathcal{N}(I_1, I_2, J)$

Verification Strategy:

Legend:

e_{tol} : Maximum allowable nodal displacement error for periodic face matching.

ϵ : Numerical perturbation step for the central finite-difference approximation.

1. Kinematic Exactness (Geometry Check)

- Purpose: Verify Netgen face-matching algorithms satisfy periodic constraints to prevent artificial boundary stiffening.
- Setup: RVE patch loaded with dummy deformation gradient $\mathbf{F} = \mathbf{1} \cdot \mathbf{1I}$.
- Check / Expected: Opposing boundary nodes strictly map to: $\mathbf{u}^+ - \mathbf{u}^- = (\mathbf{F} - \mathbf{I})(\mathbf{x}^+ - \mathbf{x}^-)$
- Pass target: Max displacement error $e_{tol} < 10^{-12}$ mm.

2. Macrohomogeneity Check (Thermodynamic Check)

- Purpose: Verify that microscopic internal energy perfectly balances macroscopic traction work.
- Setup: Fully non-linear microscopic stretch applied to the RVE.
- Check / Expected: Hill-Mandel condition equivalence. Eliminates volumetric locking in the nearly-incompressible PDMS matrix.
- Pass Target: Energy deviation < 1%.

3. Tangent Finite Difference Check (Calculus Check)

- Purpose: Prove PyTorch computational graph calculates $\hat{\mathbf{A}}$ exactly, independent of training.
- Setup: Feed stretch tensor $\mathbf{F}$, extract $\widehat{\mathbf{A}_{autograd}}$. Perturb tensor $\mathbf{F} \pm \epsilon$.
- Check / Expected: $\widehat{\mathbf{A}_{autograd}}$ strictly matches $\frac{\hat{\mathbf{P}}(\mathbf{F} + \epsilon) - \hat{\mathbf{P}}(\mathbf{F} - \epsilon)}{2\epsilon}$.
- Pass Target: Relative error drops to machine precision at $\epsilon \rightarrow 10^{-7}$.

Verification Strategy:

4. Single-Element UMAT Benchmark (Assembly Check)

- Purpose: Verify manual global stiffness assembly and Newton-Raphson convergence rates.
- Setup: Single macroscopic hexahedral element using the PyTorch UMAT under uniform boundary loading.
- Check / Expected: Custom Python loop correctly updates displacement via $\mathbf{u}_{k+1} = \mathbf{u}_k - \mathbf{K}_k^{-1} \mathbf{R}_k$ using the exact AI tangent $\mathbf{K}_k$.
- Pass Target: Strict asymptotic quadratic convergence of the global residual norm $\|\mathbf{R}\|$ ($10^{-1} \rightarrow 10^{-3} \rightarrow 10^{-8}$).

Legend:

$\mathbf{u}_k$: Global macroscopic displacement vector at Newton iteration k .

$\mathbf{K}_k$: Global assembled tangent stiffness matrix.

$\mathbf{R}_k$: Global macroscopic residual vector.

$\|\mathbf{R}\|$: L2-Norm of the global residual.

Schedule

Week 1 - 3

M1: Software Architecture, Environment Initialization, RVE Geometry & Periodic Boundary Condition (PBC) Enforcer

- Establish Python 3.10 virtual environment and continuous integration pipeline.
- Initialize GitHub repository following standard package architecture.
- Configure and test NGSolve, Netgen, and PyTorch dependencies.
- Script automated Netgen PDMS-glass bead microstructures.
- Formulate and code strict kinematic node-matching algorithms.
- Diagnostic check: Execute V1 (Kinematic Exactness).

Week 4 - 7

M2: Microscopic Homogenization & Data Generation

- Implement Hill-Mandel volume averaging scripts.
- Extract large-scale $(\bar{\mathbf{F}}, \bar{\mathbf{P}}, \bar{\mathbf{A}})$ tensor tuples across varying volume fractions.
- Diagnostic check: Execute V2 (Macrohomogeneity Check).

Week 8 - 11

M3: pICNN Architecture Implementation & Sobolev Training

- Code the PyTorch forward pass with softplus weight constraints.
- Implement thermodynamic normalization and the Sobolev loss function.
- Train surrogate and evaluate loss landscape.

Week 12 - 14

M4: Algorithmic UMAT Bridge & Tensor Mapping

- Code the dynamic integration point extraction from NGSolve.
- Implement torch.autograd.grad chains to extract analytical tangents.
- Flatten $3 \times 3 \times 3 \times 3$ tensors to 6×6 Voigt notation arrays.
- Diagnostic check: Execute V3 (Finite Difference Check).

Week 15 - 17

M5: Monolithic Macroscopic FEM Implementation

- Construct the macroscopic non-linear boundary value problem (BVP).
- Code the custom Python-based Newton-Raphson update loop $\mathbf{u}_{k+1} = \mathbf{u}_k - \mathbf{K}_k^{-1} \mathbf{R}_k$.
- Bypass NGSolve native material libraries to assemble the global stiffness matrix manually.

Week 18 - 19

M6: Single-Element Benchmark & Component Simulation

- Deploy the trained PyTorch UMAT onto a macroscopic hexahedral element.
- Diagnostic check: Execute V4 (Assembly Check) to confirm asymptotic quadratic convergence.
- Run a multi-element validation test (e.g., tensile dogbone).

Week 20 - 22

M7: Final Demonstration, Documentation, and Presentation

- Compile the final LaTeX report and finalize repository structure.
- Finalize README.md files with execution commands for the V-Suite test directories.
- Prepare final defense slides and rehearse the live code demonstration.